Projeto e Análise de Algoritmos
Trabalho 4

Gideão Santana

1 Questão 1 – Empacotamento de livros em envelopes (Algoritmo Guloso)

Notação. Seja S o conjunto de livros ainda não empacotados, $C = 5$ a capacidade de um envelope, $h \in S$ um livro de *maior* peso em S e $\ell \in S \setminus \{h\}$ um livro de *menor* peso em $S \setminus \{h\}$. Um envelope é *viável* se contém no máximo dois livros e peso total $\leq C$. Denotamos por $\mathrm{OPT}(S)$ o número mínimo de envelopes para S.

(a) Estratégia gulosa

Ordene os livros em ordem não crescente de peso. A cada passo, tome o livro mais pesado ainda não empacotado, h:

- se $p_h + p_\ell \leq 5$, coloque h e o livro mais leve restante ℓ no mesmo envelope, $\{h, \ell\}$;
- caso contrário (ou se h for o único livro restante), coloque h sozinho, $\{h\}$.

Por que é promissora. O livro mais pesado é o mais difícil de acomodar: se ele não cabe com o livro mais leve, não cabe com nenhum outro, e portanto ficará sozinho em *qualquer* solução. Se cabe, usar o mais leve como parceiro é a escolha que menos "gasta" livros úteis: os livros de peso intermediário ficam disponíveis para serem pareados entre si ou com outros livros pesados. **Exemplo.** Ordenando: $5\,(4.0), 3\,(3.0), 4\,(2.5), 2\,(2.0), 1\,(1.0)$. Obtemos $\{5, 1\}$ $(4 + 1 = 5)$, $\{3, 2\}$ $(3 + 2 = 5)$ e $\{4\}$: 3 envelopes.

(b) Propriedade de escolha gulosa

Lema 1 (Escolha gulosa)**.** *Seja S não vazio, h um livro de maior peso em S e ℓ um livro de menor peso em $S \setminus \{h\}$. Seja E o envelope escolhido pelo algoritmo, isto é, $E = \{h, \ell\}$ se $|S| \geq 2$ e $p_h + p_\ell \leq 5$, e $E = \{h\}$ caso contrário. Então existe uma solução ótima para S que contém o envelope E.*

Demonstração. Seja O uma solução ótima qualquer para S.

Caso (ii): $E = \{h\}$. Se $|S| = 1$, é trivial. Senão, $p_h + p_\ell > 5$, e para todo $x \in S \setminus \{h\}$ temos $p_x \geq p_\ell$, logo $p_h + p_x \geq p_h + p_\ell > 5$. Assim nenhum livro pode acompanhar h, e em O (como em qualquer solução) h está sozinho: $\{h\} \in O$.

Caso (i): $E = \{h, \ell\}$, com $p_h + p_\ell \leq 5$. Se $\{h, \ell\} \in O$, nada a fazer. Caso contrário, construímos O' por troca:

1. *h está sozinho em O.* Seja B o envelope de ℓ em O ($B = \{\ell\}$ ou $B = \{\ell, y\}$). Substitua $\{h\}$ e B por $\{h, \ell\}$ e $B \setminus \{\ell\}$ (descartando-o se ficar vazio). $\{h, \ell\}$ é viável por hipótese e $B \setminus \{\ell\}$ é viável por ser subconjunto de envelope viável.
2. *h está com $x \neq \ell$ em O, ou seja, $\{h, x\} \in O$.* Seja B o envelope de ℓ.

- Se $B = \{\ell\}$: troque $\{h, x\}, \{\ell\}$ por $\{h, \ell\}, \{x\}$.
- Se $B = \{\ell, y\}$: troque $\{h, x\}, \{\ell, y\}$ por $\{h, \ell\}, \{x, y\}$. O envelope $\{x, y\}$ é viável, pois $p_y \leq p_h$ (h é o mais pesado) e, portanto, $p_x + p_y \leq p_x + p_h \leq 5$.

Em todos os casos O' é uma partição viável de S com $|O'| \leq |O|$, logo O' é ótima e contém $\{h, \ell\}$. □

(c) Propriedade de subestrutura ótima

O subproblema resultante após a escolha gulosa E é $S' = S \setminus E$:

- caso (i): $S' = S \setminus \{h, \ell\}$ (os livros restantes, exceto o mais pesado e o mais leve);
- caso (ii): $S' = S \setminus \{h\}$ (os livros restantes, exceto o mais pesado).

Em ambos os casos S' é uma instância do mesmo problema (mesmas restrições).

Lema 2 (Subestrutura ótima)**.** *Se O é uma solução ótima para S que contém E (existe pelo Lema 1), então $O \setminus \{E\}$ é uma solução ótima para S'. Consequentemente,* $\mathrm{OPT}(S) = 1 + \mathrm{OPT}(S')$*, e $\{E\} \cup O^*$ é ótima para S para qualquer solução ótima O^* de S'.*

Demonstração. $O \setminus \{E\}$ particiona S' em envelopes viáveis, com $|O| - 1$ envelopes. Suponha que exista uma solução O'' para S' com $|O''| < |O| - 1$. Então $O'' \cup \{E\}$ é uma solução viável para S (pois E é viável e disjunto de S') com $|O''| + 1 < |O|$ envelopes, contradizendo a otimalidade de O. Logo $\mathrm{OPT}(S') = |O| - 1 = \mathrm{OPT}(S) - 1$, e para qualquer O^* ótima de S', $|\{E\} \cup O^*| = 1 + \mathrm{OPT}(S') = \mathrm{OPT}(S)$. □

(d) Pseudocódigo

Após ordenar em ordem não crescente, o índice i aponta para o livro mais pesado restante e j para o mais leve restante (técnica de dois ponteiros).

Algoritmo 1 EMPACOTA-LIVROS(p, n)

Entrada: pesos $p_1, \ldots, p_n$, com $0 < p_k \leq 5$
Saída: lista $\mathcal{E}$ de envelopes com o menor número possível de envelopes

```
1:  L ← livros 1..n ordenados em ordem não crescente de peso        ▷ O(n lg n)
2:  E ← ∅
3:  i ← 1;   j ← n
4:  while i ≤ j do
5:      if i < j and p_{L[i]} + p_{L[j]} ≤ 5 then                   ▷ caso (i)
6:          E ← E ∪ {{L[i], L[j]}}
7:          j ← j − 1
8:      else                                                        ▷ caso (ii) ou último livro
9:          E ← E ∪ {{L[i]}}
10:     i ← i + 1
11: return E
```

Invariante: no início de cada iteração, os livros não empacotados são exatamente $L[i..j]$, com $L[i]$ o mais pesado e $L[j]$ o mais leve entre eles.

(e) Correção

Teorema 1. *EMPACOTA-LIVROS devolve uma distribuição viável com* $\mathrm{OPT}(S)$ *envelopes.*

Demonstração. Por indução em $|S|$ (o número de livros em $L[i..j]$). *Base:* $|S| = 0$: o algoritmo devolve $\emptyset$, que é ótimo. *Passo:* seja $|S| \geq 1$. Pelo invariante, o algoritmo escolhe exatamente o envelope E dos itens (a)–(b): $\{L[i], L[j]\}$ se cabem juntos e $i < j$, ou $\{L[i]\}$ caso contrário; E é viável. Pelo Lema 1, existe solução ótima de S que contém E. O algoritmo prossegue sobre $S' = S \setminus E = L[i'..j']$ (subproblema identificado em (c)), com $|S'| < |S|$; pela hipótese de indução, devolve uma solução ótima O^* de S'. Pelo Lema 2, $\{E\} \cup O^*$ é ótima para S, com $1 + \mathrm{OPT}(S') = \mathrm{OPT}(S)$ envelopes. Como cada livro entra em exatamente um envelope (cada índice sai de $L[i..j]$ uma única vez), a saída é uma partição válida. □

(f) Complexidade

- **Ordenação:** $O(n \lg n)$ com um algoritmo ótimo por comparação (*Merge Sort* ou *Heap-sort*).
- **Laço:** a cada iteração i aumenta em 1 (e j possivelmente diminui em 1), logo $j - i$ decresce ao menos 1 por iteração e há no máximo n iterações, cada uma em $O(1)$: total $O(n)$.
- **Construção da saída:** cada envelope é inserido em $O(1)$; são no máximo n envelopes: $O(n)$.

Portanto, $T(n) = O(n \lg n) + O(n) + O(n) = O(n \lg n)$, dominado pela ordenação. O espaço adicional é $O(n)$.

2 Questão 2 – Paradas em hotéis (Algoritmo Guloso)

Notação. Uma sequência de hotéis $S = [s_1, \ldots, s_r]$ é *viável* se $s_1 = 1$, $s_r = n$, $s_i < s_{i+1}$ e $x[s_{i+1}] - x[s_i] \leq D$ para todo $1 \leq i < r$. A escolha gulosa a partir da parada u é $\max\{\, w > u : x[w] - x[u] \leq D \,\}$, que existe para $u < n$, pois $x[u+1] - x[u] \leq D$.
Exemplo ($D = 12$): a partir do hotel 1 ($x = 0$), o mais distante alcançável é o hotel 4 ($x = 11$); a partir do hotel 4, é o hotel 6 ($x = 17$). Logo $G = [1, 4, 6]$.

(a) Propriedade de escolha gulosa (argumento de troca)

Sejam $G = [u_1, \ldots, u_m]$ e $O = [v_1, \ldots, v_\ell]$ como no enunciado, com $v_i = u_i$ para $1 \leq i < k$, $v_k \neq u_k$ e $1 < k < \ell$.

Lema 1. $v_k < u_k$.

Demonstração. Como O é viável e $v_{k-1} = u_{k-1}$, temos $x[v_k] - x[u_{k-1}] \leq D$ e $v_k > u_{k-1}$. Logo v_k é um candidato da escolha gulosa a partir de u_{k-1}, e u_k é o maior candidato: $v_k \leq u_k$. Como $v_k \neq u_k$, $v_k < u_k$. □

Construção. Troque v_k por u_k em O, descartando os hotéis de O que ficaram "para trás" de u_k:

$$O' = [\, v_1, \ldots, v_{k-1},\ u_k,\ v_{j+1}, \ldots, v_\ell \,], \qquad j = \max\{\, i \geq k : v_i \leq u_k \,\}.$$

(Note que $j \geq k$, pois $v_k < u_k$. Se $v_j = u_k$, então v_j e u_k são o mesmo hotel e ele aparece uma única vez.)

Lema 2. *O' é viável e $|O'| \leq |O|$.*

Demonstração. Tamanho: O' remove de O os $j - k + 1 \geq 1$ hotéis $v_k, \ldots, v_j$ e insere apenas u_k, logo $|O'| = \ell - (j - k) \leq \ell$.

Ordem: $v_{k-1} = u_{k-1} < u_k$ e, pela escolha de j, $u_k < v_{j+1}$.

Distâncias: os trechos $v_1 \to \cdots \to v_{k-1}$ e $v_{j+1} \to \cdots \to v_\ell$ são trechos de O, portanto $\leq D$. Restam dois trechos:

- $u_{k-1} \to u_k$: $x[u_k] - x[u_{k-1}] \le D$ pela definição da escolha gulosa;
- $u_k \to v_{j+1}$ (se $j < \ell$): como $x[v_j] \le x[u_k]$,

$$x[v_{j+1}] - x[u_k] \;\le\; x[v_{j+1}] - x[v_j] \;\le\; D.$$

Extremos: O' começa em $v_1 = 1$. Se $j < \ell$, termina em $v_\ell = n$; se $j = \ell$, então $n = v_\ell \le u_k \le n$, logo $u_k = n$ e O' termina em n. □

Como O é ótima e $|O'| \le |O|$, O' também é ótima (logo, na verdade, $j = k$ e $|O'| = |O|$), e O' coincide com G nas k primeiras paradas. Isso prova a sentença: é possível trocar v_k por u_k sem piorar a solução ótima.

Teorema 1. *A solução gulosa G é ótima, ou seja, $m = \ell$.*

Demonstração. Aplicando a troca repetidamente, o primeiro índice de divergência cresce a cada passo; obtemos assim uma solução ótima $O^* = [v_1, \ldots, v_\ell]$ com $v_i = u_i$ para todo $1 \le i \le \ell - 1$. Como O^* é viável, $x[n] - x[u_{\ell-1}] = x[v_\ell] - x[v_{\ell-1}] \le D$; então o hotel n (o mais distante de todos) é alcançável a partir de $u_{\ell-1}$ e a escolha gulosa é $u_\ell = n$. Assim o guloso termina com $m = \ell$ paradas. (Equivalentemente, pelo subproblema descrito no enunciado, a cada troca o problema restante parte de $x[u_k]$ e o argumento se repete por indução.) □

(b) Pseudocódigo e complexidade

Algoritmo 2 PARADAS-HOTÉIS(x, n, D)

Entrada: posições $x[1..n]$ em ordem crescente, com $x[i+1] - x[i] \le D$
Saída: lista G com o menor número de paradas, de 1 até n

```
1: G ← [1]
2: u ← 1                                          ▷ última parada
3: i ← 1                                          ▷ candidato atual
4: while u < n do
5:     while i < n and x[i + 1] − x[u] ≤ D do
6:         i ← i + 1                              ▷ avança enquanto o próximo hotel é alcançável
7:     u ← i                                      ▷ escolha gulosa: hotel mais distante alcançável
8:     ANEXA(G, u)
9: return G
```

Terminação e corretude do laço. No início de cada iteração do laço externo, $i = u$. O laço interno avança i ao maior índice com $x[i] - x[u] \le D$ (a escolha gulosa); como $x[u+1] - x[u] \le D$, ele avança pelo menos uma vez, logo u cresce estritamente e o algoritmo termina com $u = n$.
Complexidade. O índice i nunca decresce e vai de 1 até n, então o laço interno executa no total no máximo $n - 1$ incrementos ao longo de toda a execução. O laço externo executa no máximo $n - 1$ vezes, cada uma com custo $O(1)$ fora do laço interno (anexar a G custa $O(1)$). Logo o tempo total é

$$T(n) = O(n),$$

sem necessidade de ordenação, pois x já está em ordem crescente. O espaço adicional é $O(n)$ para armazenar G (ou $O(1)$, se as paradas forem apenas impressas).

3 Questão 3 – Impressão nítida (Programação Dinâmica)

Ideia. No algoritmo recursivo, a única informação que muda entre as chamadas é i, o índice da primeira palavra do subproblema (M e W são fixos). Como o mesmo i é resolvido muitas vezes por caminhos diferentes de quebras, memorizamos a resposta de cada i em vetores:

- *custo*[i]: custo mínimo para imprimir as palavras $i, \ldots, n$;
- *prox*[i]: índice j da próxima quebra de linha em uma solução ótima a partir de i (*prox*[i] $= n+1$ indica que não há quebra).

A recorrência, tirada diretamente das linhas 01–11 do algoritmo recursivo, é

$$\mathit{custo}[i] = \begin{cases} 0, & \text{se } M[i,n] \le W, \\ \min\limits_{\substack{i<j\le n \\ M[i,j-1]\le W}} \Big\{(W - M[i,j-1])^2 + \mathit{custo}[j]\Big\}, & \text{caso contrário.} \end{cases}$$

O conjunto do min nunca é vazio, pois $M[i,i] = L[i] \le W$ (o candidato $j = i+1$ sempre é válido). A resposta do problema é *custo*[1].

(a) Número de subsoluções

São armazenadas n subsoluções: uma para cada $i \in \{1, \ldots, n\}$, isto é, *custo*[1..n] (e o respectivo *prox*[1..n]). *Justificativa:* todo subproblema gerado pela recursão é "imprimir as palavras $i, \ldots, n$", identificado unicamente pelo parâmetro i, que só assume valores entre 1 e n (a chamada inicial usa $i = 1$ e as recursivas usam $j \in \{i+1, \ldots, n\}$). Portanto há apenas n subproblemas distintos, embora a recursão sem memorização os resolva um número exponencial de vezes.

(b) Caso base

O caso base corresponde às linhas 01–02: para todo i tal que $M[i,n] \le W$, as palavras $i, \ldots, n$ cabem em uma única linha, que é a última e tem custo zero. Assim,

$$\mathit{custo}[i] = 0 \quad \text{e} \quad \mathit{prox}[i] = n+1 \qquad \text{para todo } i \text{ com } M[i,n] \le W.$$

Como $M[i,n]$ decresce quando i cresce, esses índices formam um sufixo $\{i_0, \ldots, n\}$; em particular, $i = n$ é sempre caso base, pois $M[n,n] = L[n] \le W$.

(c) Ordem de preenchimento

A tabela é preenchida com i **decrescente**, de n até 1.

Argumento: *custo*[i] depende apenas de *custo*[j] com $j > i$ (na recorrência, j varia de $i+1$ a n). Quando calculamos a posição i, todas as posições $i+1, \ldots, n$ já foram preenchidas, seja pelo caso base, seja em iterações anteriores. Logo nenhum elemento ainda não preenchido é acessado. A matriz M é preenchida antes, pelo algoritmo do enunciado.

(d) Pseudocódigo

Algoritmo 3 IMPRESSÃO-NÍTIDA-PD(L, n, W)

Entrada: tamanhos $L[1..n]$, com $L[i] \leq W$, e largura W
Saída: custo mínimo e conjunto de quebras de uma solução ótima

```
 1: for i ← 1 até n do                                   ▷ preenche M em O(n²)
 2:     M[i, i] ← L[i]
 3:     for j ← i + 1 até n do
 4:         M[i, j] ← M[i, j − 1] + L[j] + 1
 5: for i ← n até 1 passo −1 do                          ▷ ordem decrescente
 6:     if M[i, n] ≤ W then                              ▷ caso base: última linha
 7:         custo[i] ← 0;    prox[i] ← n + 1
 8:     else
 9:         custo[i] ← ∞
10:         for j ← i + 1 até n do
11:             if M[i, j − 1] > W then break
12:             c ← (W − M[i, j − 1])² + custo[j]
13:             if c < custo[i] then
14:                 custo[i] ← c;    prox[i] ← j
15: sol ← ∅;    i ← 1                                   ▷ reconstrução da solução
16: while prox[i] ≤ n do
17:     i ← prox[i]
18:     sol ← sol ∪ {i}
19: return (sol, custo[1])
```

Exemplo. Para a entrada do enunciado ($W = 12$), o algoritmo obtém $custo[1] = 33$ com quebras em $\{3, 5, 8, 11\}$. Logo, a solução apresentada no enunciado é ótima.

(e) Complexidade de tempo

- **Preenchimento de M:** $\sum_{i=1}^{n}(n - i + 1) = O(n^2)$.
- **Preenchimento de *custo* e *prox*:** são n subproblemas (item (a)); cada um examina no máximo $n - i$ candidatos j, cada um em $O(1)$ (a linha 10 do recursivo vira a atribuição $prox[i] \leftarrow j$, de custo constante). Total: $\sum_{i=1}^{n} O(n - i) = O(n^2)$.
- **Reconstrução:** cada passo avança i estritamente, logo $O(n)$.

Portanto, o tempo total é

$$T(n) = O(n^2) + O(n^2) + O(n) = O(n^2),$$

polinomial, em contraste com o tempo exponencial do backtracking. O espaço é $O(n^2)$ por causa de M (os vetores *custo* e *prox* ocupam $O(n)$).